# PostgreSQL Many-to-Many Relationships

Interview-Oriented Practice Set • 5 Combined Easy → Medium → Hard Questions

**Important constraints:** Use PostgreSQL only. Do not use subqueries, CTEs, or window functions. Solve using JOINs, GROUP BY, HAVING, aggregate functions, CASE, WHERE, ORDER BY, and related concepts.

**Goal:** Strengthen many-to-many relationship logic through junction tables and prepare for SQL coding interviews.

## 1. Database Setup

The following three tables model a many-to-many relationship: one student can enroll in many courses, and one course can have many students. The **enrollments** table is the junction/bridge table.

### students

| student_id | student_name | department |
|---|---|---|
| 1 | Aarav | Computer |
| 2 | Priya | IT |
| 3 | Rahul | Computer |
| 4 | Sneha | ENTC |
| 5 | Vikram | IT |
| 6 | Ananya | Computer |

### courses

| course_id | course_name | category | credits |
|---|---|---|---|
| 101 | PostgreSQL | Database | 4 |
| 102 | Python | Programming | 4 |
| 103 | Machine Learning | AI | 5 |
| 104 | Web Development | Development | 3 |
| 105 | Data Structures | Programming | 4 |

### enrollments (junction table)

| student_id | course_id | enrollment_status | marks |
|---|---|---|---|
| 1 | 101 | Completed | 88 |
| 1 | 102 | Completed | 91 |
| 1 | 103 | Completed | 79 |
| 2 | 101 | Completed | 84 |
| 2 | 104 | Completed | 76 |
| 3 | 101 | Completed | 95 |
| 3 | 102 | Completed | 87 |
| 3 | 105 | Completed | 82 |
| 4 | 103 | Completed | 72 |
| 4 | 104 | Completed | 81 |
| 5 | 102 | Completed | 89 |
| 5 | 105 | Completed | 93 |
| 6 | 101 | Completed | 90 |
| 6 | 103 | Completed | 94 |
| 6 | 105 | Completed | 86 |

## Q1 — Easy: Student Course Enrollment Count

**Task:** Write a query to display every student along with the number of courses they are enrolled in.

• Output columns: student_id, student_name, total_courses.

• Include students even if they have zero enrollments.

• Sort by total_courses in descending order and then student_name alphabetically.

• Concepts tested: LEFT JOIN, COUNT, GROUP BY, ORDER BY.

## Q2 — Easy/Medium: Course Popularity

**Task:** Find how many students are enrolled in each course.

• Output: course_id, course_name, category, total_students.

• Include courses with zero students.

• Show the most popular course first.

• Concepts tested: LEFT JOIN, COUNT, GROUP BY, sorting.

## Q3 — Medium: Student Performance by Course

**Task:** For each course, calculate the average marks, highest marks, lowest marks, and number of completed enrollments.

• Output: course_name, average_marks, highest_marks, lowest_marks, completed_students.

• Only completed enrollments should contribute to the calculations.

• Round average_marks to 2 decimal places.

• Show only courses whose average marks are at least 80.

• Sort by average_marks descending.

• Concepts tested: JOIN, WHERE, AVG, MAX, MIN, COUNT, GROUP BY, HAVING.

## Q4 — Medium/Hard: Student Course Performance Summary

**Task:** Create a performance summary for each student based on all of their enrolled courses.

• Output: student_name, total_courses, average_marks, highest_marks, lowest_marks.

• Calculate average, highest, and lowest marks from their enrollments.

• Include every student.

• Round average_marks to 2 decimal places.

• Display only students whose average marks are 85 or higher.

• Sort by average_marks descending.

• Concepts tested: many-to-many JOIN, aggregate functions, GROUP BY, HAVING.

## Q5 — Hard: Course Category + Performance Analysis

**Task:** Analyze course participation and performance by category. For each course, show its student count, average marks, number of high scorers (marks >= 90), and a performance label.

• Output: course_name, category, total_students, average_marks, high_scorers, performance_level.

• performance_level should be: 'Excellent' when average_marks >= 90, 'Good' when average_marks >= 80, otherwise 'Needs Improvement'.

• Count high_scorers using CASE inside an aggregate function.

• Include only courses having at least 2 enrolled students.

• Sort first by category alphabetically and then average_marks descending.

• Do not use subqueries, CTEs, or window functions.

• Concepts tested: many-to-many JOIN, COUNT, AVG, CASE, GROUP BY, HAVING, ORDER BY.

## Interview Challenge Checklist

■ Can you explain why enrollments is required as a junction table?

■ Can you identify when to use INNER JOIN vs LEFT JOIN?

■ Can you explain why COUNT(column) and COUNT(*) can produce different results with LEFT JOIN?

■ Can you explain the difference between WHERE and HAVING?

■ Can you use CASE inside SUM/COUNT to calculate conditional totals?

■ Can you predict the number of rows produced by a many-to-many JOIN before grouping?

■ Can you solve these questions without subqueries, CTEs, or window functions?

**Recommended practice method:** First write the JOINs on paper, identify the row-level result, then add aggregates, GROUP BY, HAVING, and finally ORDER BY. Avoid looking up the final query until you have attempted the problem yourself.